Programmierung 2

Gedankenprotokoll

Sommersemester 2025

HTW Saar – Informatik

Dozent:
Prof. Dr. Markus Esch

Einführung

Dieses Gedankenprotokoll soll euch einen Überblick geben, was euch in der Arbeit erwartet und wie ihr euch am besten vorbereiten könnt.

Ich persönlich finde, dass es sehr schwer ist, die Arbeit zu bestehen, wenn man nur zwei Wochen zum Lernen einplant, es sei denn, man programmiert schon seit vielen Jahren. Sowohl ich als auch einige meiner Kommilitonen fanden die Arbeit ziemlich anspruchsvoll. Unser Professor hat zwar einmal gesagt, welche Themen drankommen, inklusive der Gewichtung von Plus- und Minus-Punkten, aber darauf würde ich mich nicht verlassen. In Aufgabe 1, die eine Ankreuzaufgabe ist, kann wirklich alles vorkommen.

Mein Tipp: Lernt unbedingt auch die JavaDoc und schaut euch zu jedem Thema die offizielle Dokumentation an. Viele wichtige Details stehen dort , das reduziert das Auswendiglernen. Rechnet mit einem Stapel von etwa 140 Seiten.

Zu guter Letzt: Dieses Protokoll enthält meine Gedanken und die eines guten Freundes. Wir haben uns Mühe gegeben, alles aufzuschreiben, was uns wichtig erschien. Trotzdem sind wir keine Maschinen! Verlasst euch also nicht zu 100 Prozent auf dieses Protokoll. Nutzt es als Ergänzung, macht alle Übungsaufgaben, versteht die Theorie und lernt gründlich, das ist der sicherste Weg, die Klausur zu bestehen.

1 Aufgabe (Multiple-Choice-Aufgaben)

Wenn eine falsche Antwort angekreuzt wird oder nicht alle richtigen Antworten markiert sind, gibt es keinen Punkt. Da ich die exakten Antworten nicht kenne, habe ich hier eigene Lösungen vorgeschlagen – so wie es in der echten Klausur sein könnte. Es kann also sein, dass nur eine Antwort richtig ist, zwei oder sogar alle.

1. Wann kommt eine Rekursion zu Ende?
   - ☐ Wenn der Basisfall erreicht ist.
   - ☐ Wenn der Stack überläuft.
   - ☐ Wenn der Rückgabewert null ist.
   - ☐ Wenn die Bedingung der Rekursion false wird.
   - ☐ Wenn keine weiteren Methoden mehr aufgerufen werden.
2. Wo werden die rekursiven Aufrufe gespeichert?
   - ☐ Im Heap.
   - ☐ Im Stack.
   - ☐ In globalen Variablen.
   - ☐ Im Datenbereich des Programms.
   - ☐ In der PermGen/Metaspace.
3. Was sind die Eigenschaften von inneren Klassen?
   - ☐ Haben Zugriff auf private Felder der äußeren Klasse.
   - ☐ Können static oder non-static sein.
   - ☐ Müssen in einer eigenen Datei stehen.
   - ☐ Können von Interfaces erben.
   - ☐ Dürfen nicht public sein.

4. Auf welche Variablen kann aus einer lokalen Klasse zugegriffen werden?
   - ☐ Auf lokale Variablen der Methode, wenn sie effektiv final sind.
   - ☐ Auf Parameter der Methode, wenn sie effektiv final sind.
   - ☐ Auf statische Variablen anderer Klassen.
   - ☐ Auf globale Variablen.
   - ☐ Auf lokale Variablen anderer Methoden.
5. Was sind die Eigenschaften von lokalen Klassen?
   - ☐ Sie werden innerhalb einer Methode definiert.
   - ☐ Sie können nur innerhalb des Gültigkeitsbereichs der Methode verwendet werden.
   - ☐ Sie können nicht static sein.
   - ☐ Sie dürfen Generics verwenden.
   - ☐ Sie dürfen nicht von mehreren Klassen erben.
6. Was sind die Eigenschaften von anonymen Klassen?
   - ☐ Haben keinen Klassennamen.
   - ☐ Werden direkt bei der Objekterzeugung geschrieben.
   - ☐ Können nur einmal instanziiert werden.
   - ☐ Können Konstruktorparameter verwenden.
   - ☐ Können mehrere Interfaces gleichzeitig implementieren.
7. Eigenschaften von Generics?
   - ☐ Bieten Typprüfung zur Compilezeit.
   - ☐ Unterstützen Typlöschung (Type Erasure).
   - ☐ Können keine primitiven Typen wie int direkt verwenden.
   - ☐ Reduzieren Casting zur Laufzeit.
   - ☐ Können Wildcards als Platzhalter haben.
8. Welche Objekterzeugung ist erlaubt?
   - ☐ `GenericBox<Integer> gb = new GenericBox<Integer>();`
   - ☐ `GenericBox<int> gb = new GenericBox<int>();`
   - ☐ `GenericBox gb = new GenericBox();`
   - ☐ `GenericBox<String> gb = new GenericBox<>();`
   - ☐ `GenericBox<?> gb = new GenericBox<Object>();`
9. Was sind RAW Types?
   - ☐ Generics ohne Typangabe.
   - ☐ Veraltete Schreibweise von Generics.
   - ☐ Typen, die keine Wildcards erlauben.
   - ☐ Klassen, die nicht von Object erben.
   - ☐ Datentypen für rohe Byte-Arrays.

10. Eigenschaften von Bounded Types Parametern?
    - ☐ Können mit `extends` eingeschränkt werden.
    - ☐ Können mit `super` eingeschränkt werden.
    - ☐ Können nur Referenztypen verwenden.
    - ☐ Dürfen keine Generics verschachteln.
    - ☐ Erlauben Polymorphie innerhalb der Grenzen.
11. Objekterzeugung bei Bounded Types Parametern?
    - ☐ Instanziierung ist nur möglich, wenn eine Obergrenze definiert ist.
    - ☐ `new T()` ist nicht erlaubt.
    - ☐ Man kann Objekte des Bounded Types zurückgeben.
    - ☐ Man kann Bounded Types nicht casten.
    - ☐ Wildcards können bei Objekterzeugung nicht verwendet werden.
12. Lambda-Ausdrücke, welche Schreibweise ist richtig?
    - ☐ `(x) -> x * x`
    - ☐ `x -> x * x`
    - ☐ `(x,y) -> x + y`
    - ☐ `x,y -> x + y`
    - ☐ `(int x) -> x * x`
13. Welches Funktionales Interface nimmt ein Argument und gibt kein Ergebnis zurück?
    - ☐ `Consumer<T>`
    - ☐ `Supplier<T>`
    - ☐ `Function<T,R>`
    - ☐ `Predicate<T>`
    - ☐ `BiFunction<T,U,R>`
14. Welches Funktionales Interface nimmt zwei Argumente vom Typ T und gibt T zurück?
    - ☐ `BinaryOperator<T>`
    - ☐ `BiConsumer<T,T>`
    - ☐ `BiFunction<T,T,T>`
    - ☐ `Supplier<T>`
    - ☐ `Predicate<T>`
15. Welche Ausgabe gibt folgender Code? `applyOperatorToArray(x -> x * 2);`
    - ☐ Gibt ein Array mit verdoppelten Werten zurück.
    - ☐ Gibt ein Array mit halbierten Werten zurück.
    - ☐ Gibt einen einzelnen Wert zurück.
    - ☐ Verursacht einen Compilerfehler.
    - ☐ Gibt ein leeres Array zurück.

16. Welche der folgenden Aussagen treffen auf ArrayList zu?
    - ☐ `ArrayList extends Collection`
    - ☐ `ArrayList implements List`
    - ☐ `ArrayList extends AbstractList`
    - ☐ `ArrayList implements Comparator`
    - ☐ `ArrayList ist threadsicher`
17. Welche Aussagen treffen auf List zu?
    - ☐ Ist ein Interface.
    - ☐ Enthält nur eindeutige Elemente.
    - ☐ Erlaubt Duplikate.
    - ☐ Hat eine feste Länge.
    - ☐ Erhält Elemente in Einfügereihenfolge.
18. Welche Terminal-Operationen gibt es?
    - ☐ `forEach()`
    - ☐ `map()`
    - ☐ `reduce()`
    - ☐ `collect()`
    - ☐ `filter()`
19. Welche Eigenschaften von C gibt es?
    - ☐ Prozedurale Sprache.
    - ☐ Unterstützt Klassen und Objekte.
    - ☐ Kompiliert direkt zu Maschinencode.
    - ☐ Hat Garbage Collection.
    - ☐ Unterstützt Zeigerarithmetik.
20. Was ist die Aufgabe von einem Linker?
    - ☐ Verknüpft Objektdateien zu einem Programm.
    - ☐ Übersetzt Quellcode zu Bytecode.
    - ☐ Lädt das Programm in den Speicher.
    - ☐ Optimiert den Code zur Compilezeit.
    - ☐ Verknüpft Bibliotheken zur Laufzeit.
21. Mit welchem Wort kann man eine Structure in C definieren?
    - ☐ `class`
    - ☐ `typedef`
    - ☐ `struct`
    - ☐ `enum`
    - ☐ `record`

22. Welche der folgenden Lambda-Ausdrücke sind syntaktisch korrekt?

- ☐ (a, b) -> a+b
- ☐ (a, b) -> return a+b
- ☐ (a, b) -> return a+b;
- ☐ (a, b) -> (return a+b;)
- ☐ a, b -> a+b
- ☐ int a, int b -> a+b
- ☐ a, b -> a+b;
- ☐ (int a, int b) -> a+b

23. Gegeben sei folgender Code:

```
class Container<T extends Number> {
    private T content;
    public Container(T arg) {
        content = arg;
    }
    public Container(Container<T> other) {
        this.content = other.getContent();
    }
    public T getContent() { return this.content; }
}
```

Welche der folgenden Instanziierungen sind korrekt?

- ☐ Container<Integer> e1 = new Container<Integer>(new Integer(42));
- ☐ Container<Number> e2 = new Container<Number>(c1);
- ☐ Container<Number> e3 = new Container<Integer>(c1);
- ☐ Container<Integer> e4 = new Container<Integer>(c1);
- ☐ Container<Double> e5 = new Container<Double>(3.14);

Aufgabe: C-Pointer

Gegeben sei folgender Ausschnitt eines C-Programms:

```
long l = 140732807899964;
long* pl = &l;

printf("%p\n", &*pl);
printf("%p\n", &pl);
printf("%p\n", *pl);
```

Angenommen, der Code produziert folgende Ausgabe:

```
0x7ffee9ca2b38
0x7ffee9ca2b30
0x7ffee905eb3c
```

Welche ist die Adresse der Variablen 1?

- ☐ 0x7ffee9ca2b38
- ☐ 0x7ffee9ca2b30
- ☐ 0x7ffee905eb3c

Welche Aussage beschreibt die Ausgaben korrekt?

☐ `&*pl` und `pl` liefern die gleiche Adresse.

☐ `&*pl` liefert die Adresse von l.

☐ `&pl` liefert die Adresse des Zeigers selbst.

☐ `*pl` dereferenziert den Zeiger und gibt den Wert von l.

☐ Alle drei Ausgaben sind identisch.

24. Gegeben sei folgender Code:

```
1  public class Sorter {
2      public Integer countDigits(Integer i){
3          return String.valueOf(i).length();
4      }
5      public int findMax(List<Integer> input, Function<Integer,Integer> f){
6          int max = 0;
7          for(int i = 0; i < input.size(); i++)
8              if(f.apply(input.get(i)) > f.apply(input.get(max)))
9                  max = i;
10         return max;
11     }
12     public static void main(String[] args){
13         Sorter a = new Sorter();
14         a.findMax(new ArrayList<Integer>(), a::countDigits);
15         a.findMax(new ArrayList<Integer>(), Integer::highestOneBit);
16     }
17 }
```

i. Welche Form einer Methoden-Referenz ist in Zeile 14 gegeben?

☐ Referenzen zu statischen Methoden

☐ Referenzen zu Instanz-Methoden eines gegebenen Typs

☐ Referenzen zu Instanz-Methoden eines Objekts

☐ Referenzen zu Konstruktoren

ii. Welche Form einer Methoden-Referenz ist in Zeile 15 gegeben?

☐ Referenzen zu statischen Methoden

☐ Referenzen zu Instanz-Methoden eines gegebenen Typs

☐ Referenzen zu Instanz-Methoden eines Objekts

☐ Referenzen zu Konstruktoren

25. Gegeben sei folgender Code:

```
1  List<Integer> result = Stream.of(1, 1, 3, 4)
2      .distinct()
3      .sorted()
4      .toList();
```

Welche Ausgabe erzeugt der Code?

☐ 3

☐ 6

☐ 1, 1, 3, 4

☐ 1, 3, 4

☐ Compilerfehler

2 Aufgabe (Offene Fragen)

Beantworten Sie die folgenden Fragen in ganzen Sätzen und geben Sie die Ausgabe an, falls eine entsteht. Oder so ungefähr...

Aufgabe 2.1

Was gibt folgender Code aus?

```
int result = Stream.of(1, 2, 3)
    .reduce(1, (x, y) -> x * y);

System.out.println(result);
```

Antwort: .....

Aufgabe 2.2

Welchen Wert hat `x` nach der Ausführung des folgenden Codes?

```
int main() {
    int a[5] = {10, 20, 30, 40, 50};
    int *p = a;
    int x = *(p + 2);

    printf("%d\n", x);
    return 0;
}
```

Antwort: .....

Aufgabe 2.3

Welche Ausgabe erzeugt folgender Code?

```
public class Main {
    public static void main(String[] args) {
        List<Integer> zahlen = Arrays.asList(1, 2, 3);
        boolean result = zahlen.stream()
            .anyMatch(x -> x > 2);

        System.out.println(result);
    }
}
```

Antwort: .....

Aufgabe 2.4

Beantworten Sie die folgenden Fragen in ganzen Sätzen:

a) Was ist ein **funktionales Interface** in Java?

.....

b) Was passiert, wenn eine Rekursion unendlich weiterläuft?

.....

c) Wo speichert eine Rekursion ihre aktuellen Werte ab?

.....

d) Welcher Operator liefert in C die Adresse einer Variablen?

.....

e) Welcher Operator dereferenziert in C einen Pointer?

.....

f) Welches funktionale Interface nimmt keine Argumente und gibt ein Ergebnis zurück?

.....

Aufgabe 2.5

Welche Ausgabe erzeugt folgendes Programm?

```
1  public class FaktorielleMitPrint {
2      public static int fac(int n) {
3          if (n <= 1) {
4              System.out.println("B");
5              return 1;
6          } else {
7              return n * fac(n - 1);
8          }
9      }
10
11     public static void main(String[] args) {
12         int input = 3;
13         int result = fac(input);
14         System.out.println(result);
15     }
16 }
```

Antwort: .....

Aufgabe 2.6

Welche Ausgaben erscheinen auf der Konsole, wenn folgender Code ausgeführt wird?

```
1  public class Outer {
2      public static int i = 0;
3
4      public void testLocalClass() {
5          class Inner {
6              private int i = 1;
7              public void print(int i) {
8                  System.out.println("i:␣" + i);
9                  System.out.println("i:␣" + this.i);
10                 System.out.println("i:␣" + Outer.i);
11             }
12         }
13
14         Inner inner = new Inner();
15         inner.print(2);
16     }
17
18     public static void main(String[] args) {
19         new Outer().testLocalClass();
20     }
21 }
```

Antwort: .....

3 Aufgabe (Java-Programmierung)

Gegeben: Folgendes UML-Diagramm beschreibt eine abstrakte Klasse `Shape` und eine konkrete Subklasse `Circle`.

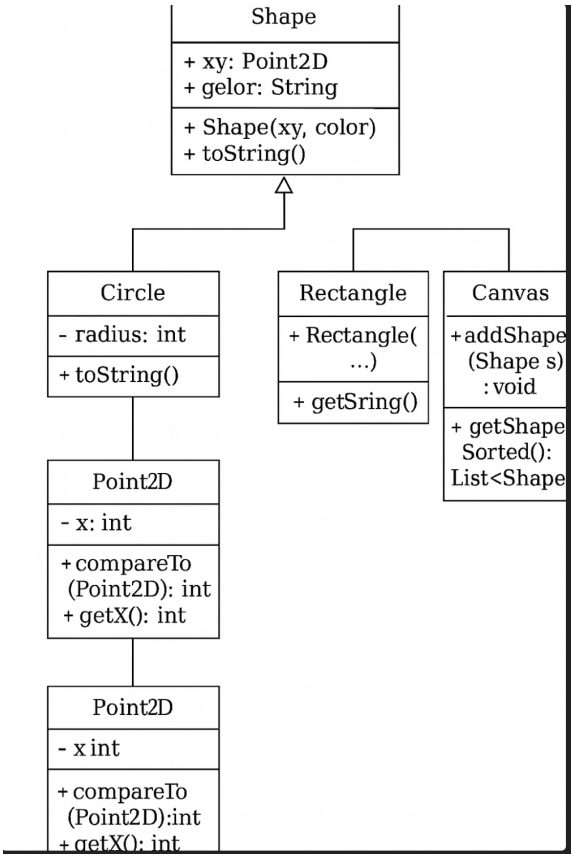


Abbildung 1: So war das... ODER SO KEIN PLAN

Implementieren Sie die folgenden Teilaufgaben:

a) Klassen-Implementierung: Implementieren Sie die Klasse `Circle`in Java.

b) Point2D Comparable machen: Implementieren Sie eine Klasse `Point2D` mit den Attributen `x` und `y`.

c) Canvas-Klasse: Implementieren Sie eine Klasse `Canvas`, die intern eine `TreeMap<Point2D, Shape>` nutzt.

4 Aufgabe (Streams und C)

Bei Aufgabe 4 ging es um **Stream-Aufgaben**. Ich empfehle, die Übungsblätter durchzuarbeiten, besonders die Aufgaben mit den **Vulkan-Daten** oder den **Schüler-Beispielen** sind sehr hilfreich und repräsentativ für die Klausur.

Zum Schluss kam noch eine **C-Aufgabe**, allerdings habe ich diese nicht mehr geschafft. Daher kann ich dazu leider keine weiteren Informationen geben.

Anmerkung zum Schluss

Ich wünsche euch allen viel Erfolg beim Lernen und beim Bestehen der Klausur!

Es wäre großartig, wenn dieses Gedankenprotokoll regelmäßig aktualisiert wird, mit neuen Fragen aus zukünftigen Klausuren, aber auch mit den alten Fragen, damit immer mehr Studierende davon profitieren können.

Egal welches Fach ihr schreibt: Wenn ihr Lust habt, schickt mir einfach eure Fragen, Übungsaufgaben oder sogar ein paar Stichpunkte als Dokument oder Sprachnachricht. Ich versuche dann, daraus eine ähnliche Klausur zu erstellen, sodass wir alle besser üben können.

Dies ist ein kleiner Appell an die Gemeinschaft, je mehr wir unser Wissen teilen, desto einfacher wird es für alle! Und falls euch dieses Protokoll wirklich geholfen hat, würde ich mich auch über eine kleine Spende freuen. UND DANKE AN **FREE FOOD** FÜR DIE HILFE!